AgroValle Connect — 2.2 Artefactos de Arquitectura de Software, Despliegue y Patrones GoF

Proyecto: AgroValle Connect (paquete `com.agrovalle.connect`) · Stack: Java 17, Spring Boot, Spring Data JPA, PostgreSQL, Maven

2.2.1 Arquitectura MVC en 4 capas

AgroValle Connect adopta una arquitectura MVC en capas sobre Spring Boot. La **Vista** corresponde a los clientes que consumen la API REST (navegador / frontend estático en `static/`), el **Controlador** es la capa de Presentación REST, y el **Modelo** se reparte en tres capas: Servicio/Dominio, Repositorio/Acceso a Datos y Persistencia física en PostgreSQL. Cada capa solo se comunica con la inmediatamente inferior.

Capa 1 — Presentación y API REST (Controlador)

Implementa controladores `@RestController` y objetos DTO. Mapea peticiones HTTP bajo `/api/v1/...`, ejecuta validaciones `@Valid` y retorna códigos de estado HTTP mediante `ResponseEntity`.

Elemento	Clase / paquete	Responsabilidad
Controladores	`AuthController`, `ProductorController`, `ProductoController`, `InicioController` (`controlador`)	Exponen `/api/v1/auth/register`, `/api/v1/productores`, `/api/v1/productos` y `/api/status`
DTO	`RegistroRequest`, `RegistroResponse`, `TipoUsuario` (`dto`)	Contratos JSON de entrada y salida; la respuesta nunca incluye la contraseña
Manejo de errores	`ManejadorErrores` (`@RestControllerAdvice`, paquete `excepcion`)	Traduce excepciones de dominio a respuestas HTTP
Códigos HTTP		201 Created (registro/creación), 200 OK (consultas), 400 Bad Request (datos inválidos), 409 Conflict (cédula o correo duplicado)

Capa 2 — Lógica de Negocio y Servicios (Modelo - Dominio)

Implementa clases `@Service` y excepciones del dominio. Aplica las reglas del negocio agrícola y administra las transacciones con `@Transactional`.

Elemento	Clase / paquete	Responsabilidad
Servicio	`RegistroService` (`servicio`)	Registro de productores (HU-01) y compradores (HU-06) dentro de una transacción
Reglas de negocio		Cédula de 6 a 10 dígitos y **no duplicada**; correo con formato válido y único; celular colombiano de 10 dígitos; contraseña de mínimo 8 caracteres con letras y números; contraseña almacenada con hash BCrypt
Excepciones de dominio	`RegistroInvalidoException` (→ 400), `RegistroDuplicadoException` (→ 409)	Señalan la violación de una regla de negocio
Entidades de dominio	`Productor`, `Producto`, `Comprador` (`modelo`)	Representan los conceptos del negocio agrícola; `Producto` referencia a su `Productor` (`@ManyToOne`)

Capa 3 — Acceso a Datos / Repositorio (Modelo - Datos)

Implementa interfaces `@Repository` que extienden `JpaRepository`, abstrayendo la comunicación SQL mediante operaciones CRUD y consultas derivadas.

Repositorio	Consultas derivadas
`ProductorRepository`	`existsByCedula(String)`
`CompradorRepository`	`existsByCorreo(String)`
`ProductoRepository`	`findByCategoria(String)`

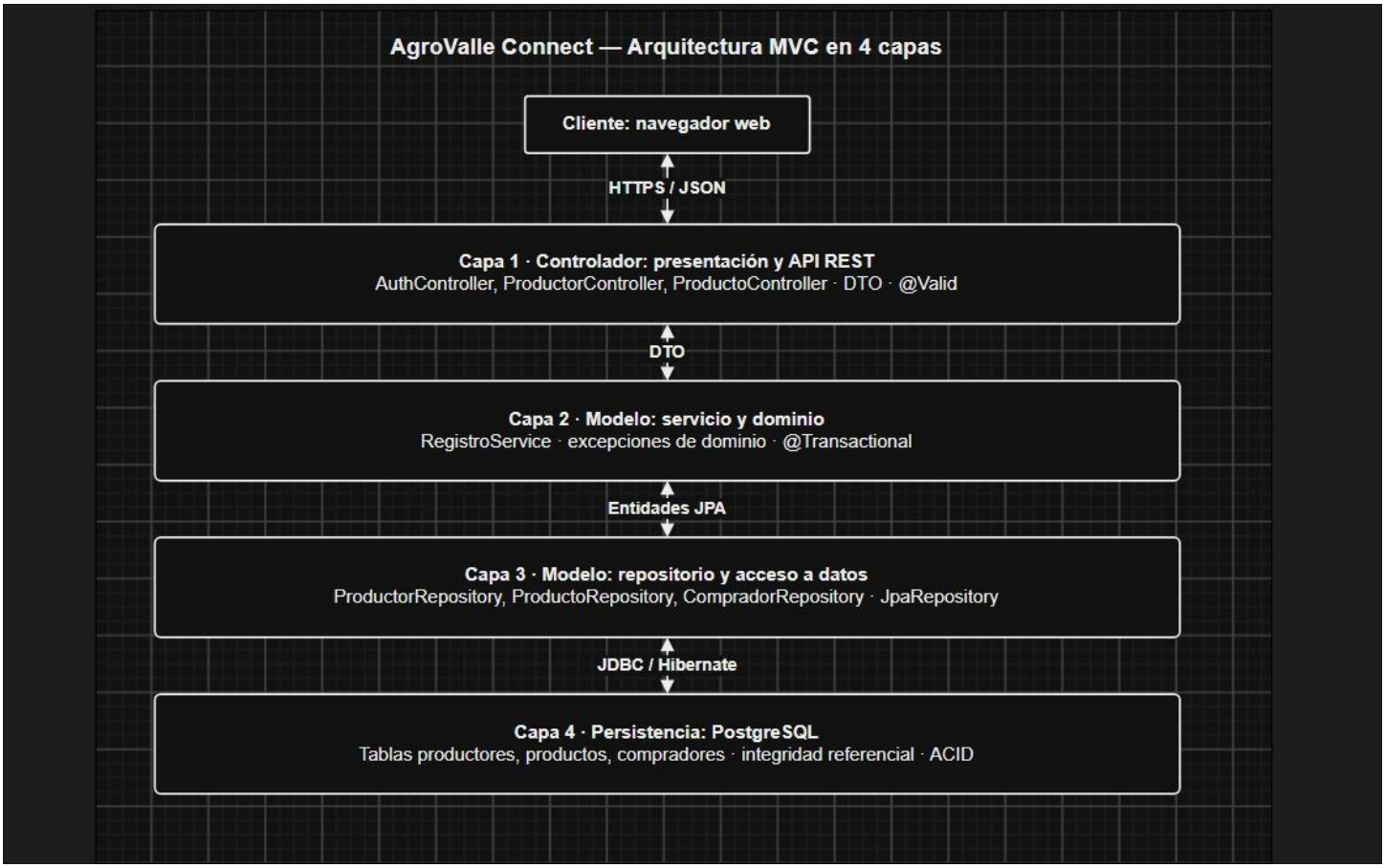
Además heredan de `JpaRepository`: `save`, `findById`, `findAll`, `deleteById`, entre otras.

Capa 4 — Persistencia Físico-Relacional (Persistencia)

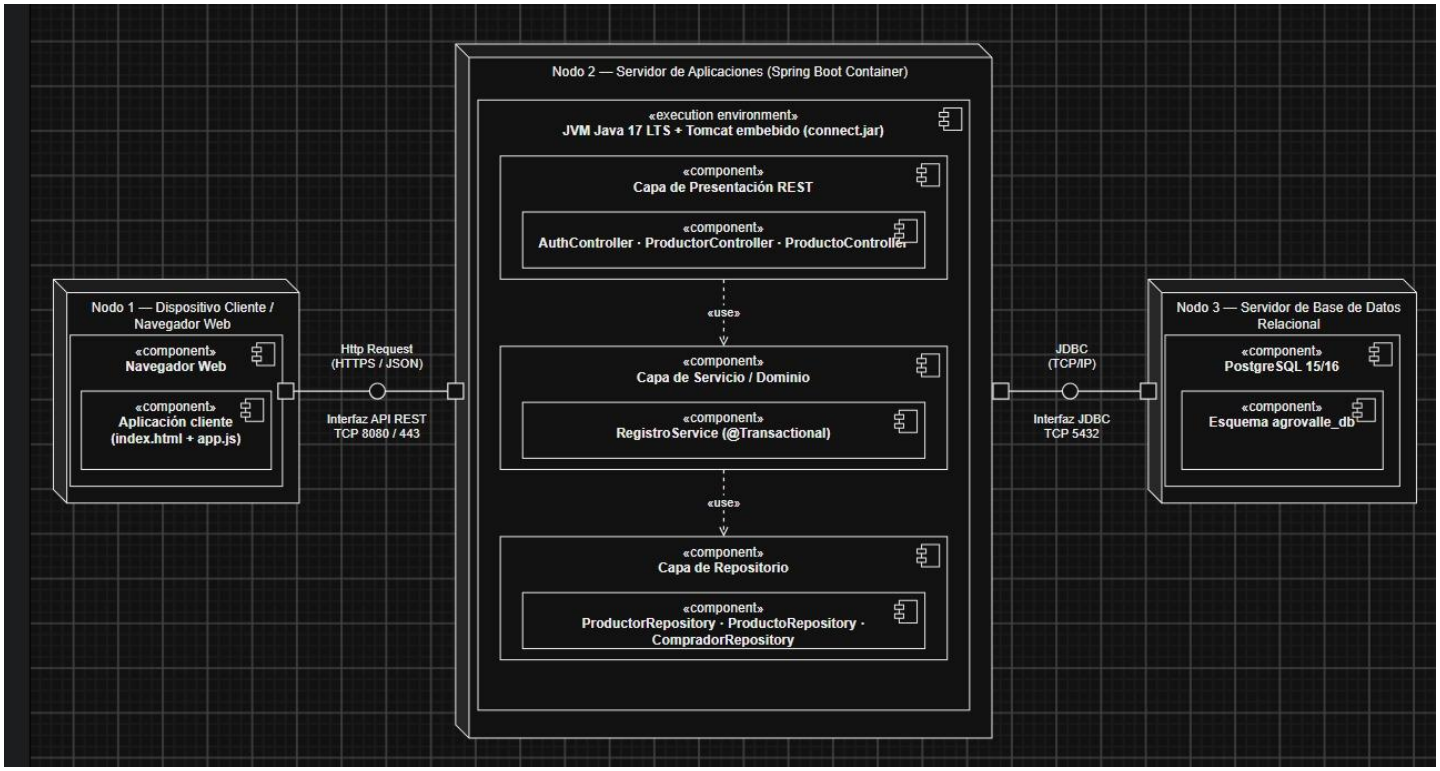
PostgreSQL almacena de forma permanente las tablas `productores`, `productos` y `compradores`, garantizando integridad referencial (clave foránea `productos.productor_id → productores.id`, restricción `UNIQUE` en `compradores.correo`) y las propiedades ACID. Hibernate genera y sincroniza el esquema (`spring.jpa.hibernate.ddl-auto=update`).

Flujo de una petición (ejemplo: registro de productor)

1. El cliente envía `POST /api/v1/auth/register` con un JSON `RegistroRequest` (tipo = PRODUCTOR).
2. `AuthController` delega en `RegistroService.registrar(...)`.
3. El servicio valida las reglas (cédula con formato válido) y consulta `ProductorRepository.existsByCedula(...)`.
4. Si la cédula ya existe lanza `RegistroDuplicadoException`; `ManejadorErrores` responde **409**. Si los datos son inválidos responde **400**.
5. Si todo es correcto, `ProductorRepository.save(...)` persiste la fila en PostgreSQL dentro de la transacción.
6. El servicio devuelve un `RegistroResponse` y el controlador responde **201 Created**.



2.2.2 Diagrama de Despliegue UML (Topología Física de Infraestructura)



2.2.3 Patrones de Diseño GoF y Enterprise

Patrón	Categoría	Propósito en AgroValle Connect	Evidencia en el código
Repository	Enterprise	Encapsula el acceso a la base de datos tras interfaces bien definidas, desacoplando el dominio del SQL	`ProductorRepository`, `ProductoRepository`, `CompradorRepository`
Singleton / Inyección de Dependencias (IoC)	GoF / Enterprise	El contenedor de Spring crea una única instancia (scope singleton) de cada Bean y administra su	Constructores de `AuthController`, `RegistroService`, `ProductorController`

Patrón	Categoría	Propósito en AgroValle Connect	Evidencia en el código
		ciclo de vida; las dependencias se inyectan por constructor	, `ProductoController`
Data Transfer Object (DTO)	Enterprise	Aísla las entidades relacionales de los contratos JSON expuestos en la API	`RegistroRequest`, `RegistroResponse` (`records` de Java)
Data Mapper / Builder	Enterprise / GoF	Facilita la conversión entre objetos DTO y entidades de dominio, construyendo cada entidad paso a paso	Conversión `RegistroRequest` → `Productor` / `Comprador` en `RegistroService`